

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Algorithm Design Challenge

COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE CODE: MLA0101
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO2: Apply search algorithms and heuristic techniques to solve state-space search and optimization problems. (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100

Time: 90 Mins

No. of Questions: 5

Annexure A Algorithm Design Challenge

Code of Conduct:

*I **Kruthika Sre S / 192525360** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Kruthika Sre

Criterion	Weightage	Marks Obtained
Problem Analysis and Understanding	15%	
AI Algorithm Selection & Justification	15%	
Algorithm Design / Pseudocode	25%	
Application of AI Concepts (Greedy, A*, CSP, Minimax, etc.)	15%	
Diagram / State Space / Search Tree Representation	10%	
Complexity Analysis and Solution Efficiency	10%	
Originality, Critical Thinking, and Presentation	10%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

Answer all the Questions

Q.No	Question	Topics
1	Design an algorithm using Greedy Best-First Search to find a path from a source node to a goal node for the given graph. Clearly define the heuristic function, show the node expansion order, and justify why Greedy Best-First Search is suitable for this problem.	Informed Search, Greedy Best-First Search, Heuristic Functions
2	A delivery robot must travel from a warehouse to a customer through a weighted road network. Design an A Search algorithm to determine the optimal path. Calculate $g(n)$, $h(n)$, and $f(n)$ for each expanded node, and explain why A guarantees the optimal solution.	A* Search, Heuristic Functions
3	A university needs to prepare an examination timetable without scheduling conflicting subjects at the same time. Design the problem as a Constraint Satisfaction Problem (CSP) by identifying variables, domains, and constraints. Explain how MRV and Forward Checking improve the efficiency of the solution.	CSP, MRV, Forward Checking
4	Design the Minimax algorithm for the given game tree to determine the best move for the MAX player. Then apply Alpha-Beta Pruning to the same tree and indicate which nodes are pruned. Compare the number of nodes evaluated before and after pruning.	Game Playing, Minimax, Alpha-Beta Pruning
5	A warehouse robot operates in a dynamic environment where obstacles may appear unexpectedly. Design an intelligent search strategy using either Local Search or an Online Search Agent. Explain how the algorithm adapts to environmental changes and compare its performance with informed search algorithms.	Local Search, Optimization, Online Search Agents

Structure of the Answer – Algorithm Design Challenge

S.No	Sections	Expected Content
1	Problem Statement	Briefly describe the given problem in your own words.
2	Problem Analysis	Identify the initial state, goal state, assumptions, and constraints.
3	AI Technique Selection	Specify the most appropriate AI algorithm/search technique (e.g., Greedy Best-First Search, A*, CSP, Minimax, Local Search).
4	Justification of Algorithm	Explain why the selected algorithm is suitable compared to other search techniques.
5	State Space Representation	Represent the problem using a state-space diagram, search graph, game tree, or constraint graph, as applicable.
6	Variables, Domains, and Constraints (Applicable for CSP problems)	Identify variables, possible domains, and constraints. If not applicable, mention "Not Applicable".
7	Heuristic Function (Applicable for Informed Search)	Define the heuristic function $h(n)$ and explain whether it is admissible or appropriate for the problem. If not applicable, mention "Not Applicable".
8	Algorithm Design / Pseudocode	Write the step-by-step algorithm or pseudocode for solving the problem.
9	Flowchart / Algorithm Diagram	Draw a flowchart, search tree, game tree, or algorithm workflow illustrating the solution.
10	Working / Execution Trace	Demonstrate the execution of the algorithm with at least one example, showing intermediate steps such as node expansion, constraint propagation, or game-tree evaluation.
11	Complexity Analysis	Analyze the Time Complexity and Space Complexity of the proposed algorithm.
12	Advantages and Limitations	Discuss the strengths and limitations of the selected algorithm for the given problem.
13	Conclusion	Summarize the effectiveness of the designed algorithm and suggest possible improvements or alternative approaches.

EVALUATION RUBRICS

Criteria	Weightage (%)	Excellent (90–100%)	Good (75–89%)	Average (60–74%)	Poor (<60%)
Problem Analysis and Understanding	15%	13–15% – Clearly identifies the problem, initial state, goal state, constraints, and expected outcome.	10–12% – Identifies most problem components with minor omissions.	7–9% – Demonstrates partial understanding with some missing elements.	0–6% – Incorrect or incomplete understanding of the problem.
AI Algorithm Selection & Justification	15%	13–15% – Selects the most appropriate AI algorithm/search technique with strong technical justification.	10–12% – Selects an appropriate algorithm with adequate justification.	7–9% – Selects a partially suitable algorithm with limited justification.	0–6% – Incorrect algorithm selection or no justification.
Algorithm Design / Pseudocode	25%	22–25% – Designs a complete, logical, efficient, and error-free algorithm/pseudocode.	17–21% – Algorithm is mostly correct with minor logical errors.	12–16% – Algorithm is partially complete with several logical errors.	0–11% – Algorithm is incomplete, incorrect, or difficult to understand.
Application of AI Concepts (Greedy, A*, CSP, Minimax, Alpha-Beta, etc.)	15%	13–15% – Correctly applies the relevant AI concepts and techniques throughout the solution.	10–12% – Applies AI concepts correctly with minor mistakes.	7–9% – Demonstrates limited application of AI concepts.	0–6% – Unable to apply the appropriate AI concepts or techniques.

Diagram / State Space / Search Tree Representation	10%	9–10% – Accurate, complete, and well-labelled diagram supporting the solution.	7–8% – Diagram is mostly correct with minor omissions.	5–6% – Diagram is partially complete or lacks clarity.	0–4% – Diagram is missing or incorrect.
Complexity Analysis and Solution Efficiency	10%	9–10% – Correctly analyzes time and space complexity and proposes an efficient solution.	7–8% – Complexity analysis is mostly correct with minor errors.	5–6% – Partial complexity analysis or limited discussion of efficiency.	0–4% – No meaningful complexity analysis or highly inefficient solution.
Originality, Critical Thinking, and Presentation	10%	9–10% – Demonstrates excellent originality, logical reasoning, organized presentation, and explores optimized/alternative solutions.	7–8% – Demonstrates good critical thinking with a clear presentation.	5–6% – Shows limited originality and acceptable presentation.	0–4% – Poor presentation with little or no critical thinking.

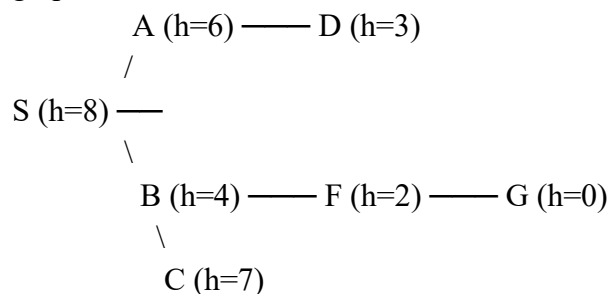
1. Design an algorithm using Greedy Best-First Search to find a path from a source node to a goal node for the given graph. Clearly define the heuristic function, show the node expansion order, and justify why Greedy Best-First Search is suitable for this problem.

Problem Statement

Design an algorithm using Greedy Best-First Search (GBFS) to find a path from a source node S to a goal node G in a given graph. The algorithm should use a heuristic function to select the node that appears closest to the goal, display the order in which nodes are expanded, and determine the resulting path.

Problem Analysis

Greedy Best-First Search is an informed search algorithm that uses only the heuristic value of a node to decide which node should be explored next. For this example, consider the following graph:



Here:

- S = Source node
- G = Goal node
- $h(n)$ = estimated cost from node n to the goal
- Lower heuristic values are preferred.

The algorithm starts from S, evaluates its neighbouring nodes, and always selects the node with the lowest heuristic value.

AI Technique Selection

The selected AI technique is Greedy Best-First Search, which is an informed search technique that uses heuristic information to guide the search toward the goal.

The evaluation function is: $f(n)=h(n)$

where $h(n)$ represents the estimated cost from node n to the goal.

Justification of Algorithm

Greedy Best-First Search is suitable because the graph provides heuristic information indicating how close each node is to the goal. Instead of exploring nodes blindly, the algorithm prioritizes the node with the smallest heuristic value. It is particularly useful when a reasonably good path needs to be found quickly and the heuristic gives meaningful guidance. However, unlike A*, GBFS does not consider the actual cost already travelled.

State Space Representation

The state space consists of all nodes that can be reached from the source node.

Node	Heuristic $h(n)$
S	8
A	6
B	4
C	7
D	3
F	2
G	0

The goal state is:

$[G, h(G)=0]$

The search begins at S and terminates when G is selected.

Variables, Domains, and Constraints

This is not a CSP problem, so variables, domains, and constraints are not applicable.

Heuristic Function

The heuristic function estimates the remaining distance from a node to the goal:

$h(n)$ = estimated cost from node n to goal G

For this example:

$h(S) = 8, h(A) = 6, h(B) = 4, h(C) = 7, h(D) = 3, h(F) = 2, h(G) = 0$

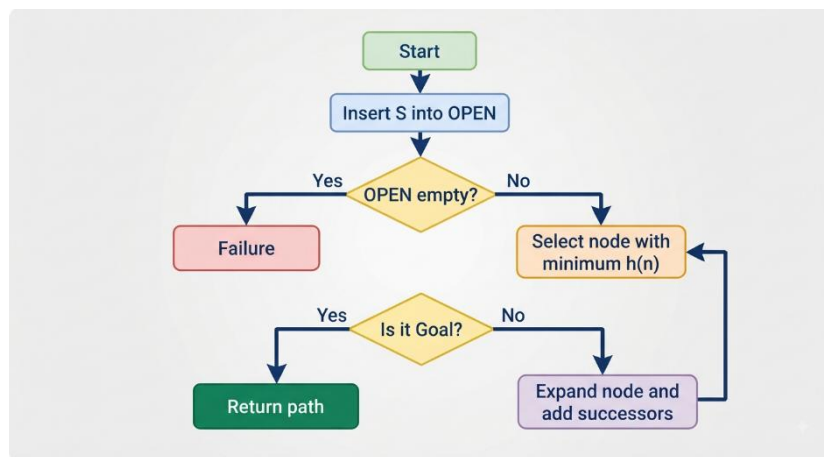
Greedy Best-First Search always selects the node having the minimum $h(n)$.

Algorithm Design / Pseudocode

GreedyBestFirstSearch(Graph, Start, Goal)

1. Create an empty priority queue OPEN.
2. Create an empty set CLOSED.
3. Insert Start into OPEN with priority $h(\text{Start})$.
4. Set $\text{parent}[\text{Start}] = \text{NULL}$.
5. While OPEN is not empty:
 - a. Remove the node n with the smallest $h(n)$.
 - b. If n is the Goal:
Reconstruct the path using parent pointers.
Return the path.
 - c. Add n to CLOSED.
 - d. For each successor m of n :
 - i. If m is not in CLOSED and not already in OPEN:
Set $\text{parent}[m] = n$.
Insert m into OPEN with priority $h(m)$.
6. Return "No path exists".

Flowchart / Algorithm Diagram



Working / Execution Trace

Starting node: S

Its successors are:

- A, $h(A) = 6$
- B, $h(B) = 4$
- C, $h(C) = 7$

The node with the smallest heuristic value is B.

Step	Expanded Node	OPEN List after Expansion	Selected Next
1	S	A(6), B(4), C(7)	B
2	B	A(6), C(7), F(2)	F
3	F	A(6), C(7), G(0)	G
4	G	Goal reached	Stop

The result order: $S \rightarrow B \rightarrow F \rightarrow G$

The search reaches the goal by repeatedly choosing the node with the smallest heuristic value.

Complexity Analysis

For b as the branching factor and d as the depth of the solution:

- Time Complexity: $(O(b^d))$ in the worst case.
- Space Complexity: $(O(b^d))$, because OPEN and CLOSED may store many generated nodes.

The actual performance depends heavily on the quality of the heuristic function.

Advantages and Limitations

Advantages:

- Uses heuristic information to guide the search.
- Usually reaches a promising goal quickly.
- Simple to implement using a priority queue.
- More directed than uninformed search methods.

Limitations:

- Does not consider the actual path cost from the source.
- Does not guarantee the shortest or optimal path.
- Can be misled by an inaccurate heuristic.
- May explore an unnecessarily large portion of the graph in the worst case.

Conclusion

Greedy Best-First Search successfully finds the path $S \rightarrow B \rightarrow F \rightarrow G$ by selecting the node with the lowest heuristic value at each step. The algorithm is suitable for problems where reaching a promising goal quickly is more important than guaranteeing the optimal path.

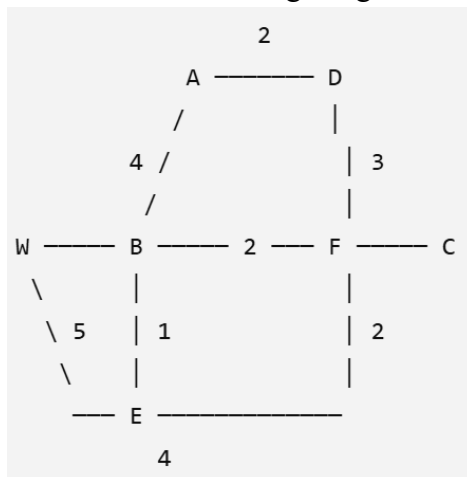
2. Design an algorithm using Greedy Best-First Search to find a path from a source node to a goal node for the given graph. Clearly define the heuristic function, show the node expansion order, and justify why Greedy Best-First Search is suitable for this problem.

Problem Statement

Design an A* Search algorithm for a delivery robot that must travel from a warehouse to a customer through a weighted road network. The algorithm should determine the optimal path by considering both the actual travel cost and the estimated remaining cost to the customer.

Problem Analysis

Consider the following weighted road network:



Let:

- W = Warehouse, source node
- C = Customer, goal node
- Edge values = actual road/travel costs
- $g(n)$ = actual cost from W to node n
- $h(n)$ = estimated cost from node n to C
- $f(n)$ = total estimated cost

The A* evaluation function is:

$$f(n) = g(n) + h(n)$$

For this example, use the following heuristic values:

Node	$h(n)$
W	8
A	7
B	6
D	5
E	5
F	3
C	0

The heuristic is assumed to be admissible, meaning it never overestimates the actual minimum remaining cost to the customer.

AI Technique Selection

The selected technique is A* Search, an informed search algorithm that combines:

- $g(n)$: actual cost travelled from the warehouse.
- $h(n)$: estimated cost remaining to the customer.

Therefore:

$$f(n) = g(n) + h(n)$$

The node with the smallest $f(n)$ is selected for expansion.

Justification of Algorithm

A* is suitable because this is a weighted path-finding problem where the robot must find the least-cost route. Unlike Greedy Best-First Search, which considers only $h(n)$, A* considers both the distance already travelled and the estimated distance remaining. This makes A* capable of finding an optimal path, provided the heuristic is admissible and, for graph search, consistent.

State Space Representation

Each node represents a possible location of the delivery robot, while each weighted edge represents the travel cost between two locations.

The relevant paths include:

$W \rightarrow A \rightarrow D \rightarrow F \rightarrow C$

$W \rightarrow B \rightarrow F \rightarrow C$

$W \rightarrow B \rightarrow E \rightarrow F \rightarrow C$

The objective is to find the path with the minimum total cost.

Variables, Domains, and Constraints

This is not a CSP problem, so variables, domains, and constraints are not applicable.

Heuristic Function

The heuristic function estimates the remaining cost from each node to the customer:

$h(n)$ = estimated minimum cost from n to C

The selected heuristic values are:

$h(W)=8, h(A)=7, h(B)=6, h(D)=5$

$h(E)=5, h(F)=3, h(C)=0$

The heuristic must satisfy:

$h(n) \leq h^*(n)$

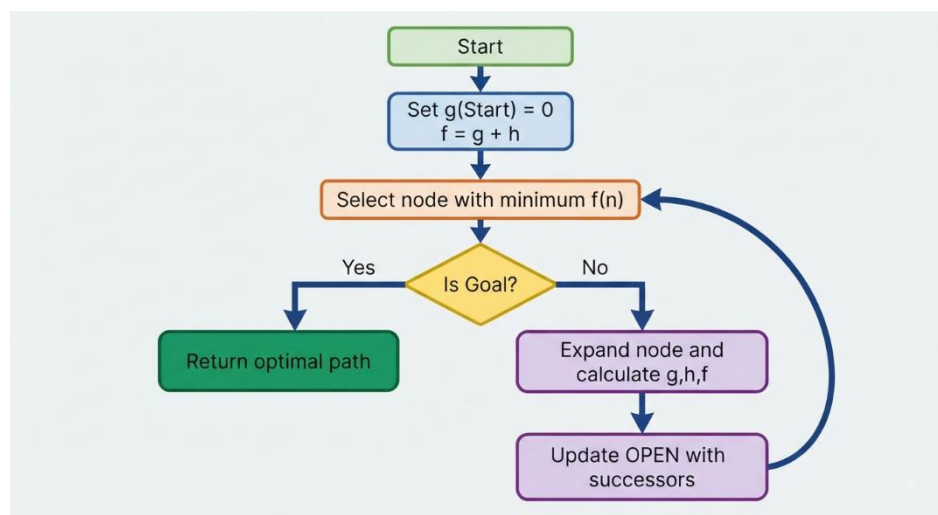
where $(h^*(n))$ is the actual minimum cost from n to the goal.

Algorithm Design / Pseudocode

AStarSearch(Graph, Start, Goal)

1. Create OPEN priority queue.
2. Create CLOSED set.
3. Set $g(\text{Start}) = 0$.
4. Calculate $f(\text{Start}) = g(\text{Start}) + h(\text{Start})$.
5. Insert Start into OPEN.
6. While OPEN is not empty:
 - a. Remove the node n with the smallest $f(n)$.
 - b. If n is Goal:
Reconstruct and return the path.
 - c. Add n to CLOSED.
 - d. For each neighbor m of n :
Calculate:
 $\text{tentative_g} = g(n) + \text{cost}(n,m)$
If m is not explored OR
 $\text{tentative_g} < g(m)$:
 $\text{parent}(m) = n$
 $g(m) = \text{tentative_g}$
 $f(m) = g(m) + h(m)$
Insert/update m in OPEN.
7. Return "No path exists".

Flowchart / Algorithm Diagram



Working / Execution Trace

Start with:

$$g(W)=0$$

$$f(W)=g(W)+h(W)=0+8=8$$

Expand W. Its neighbours are A and B.

For A:

$$g(A)=4$$

$$f(A)=4+7=11$$

For B:

$$g(B)=5$$

$$f(B)=5+6=11$$

Both have the same $f(n)$. Select B.

Expand B:

For E:

$$g(E)=5+1=6$$

$$f(E)=6+5=11$$

For F:

$$g(F)=5+2=7$$

$$f(F)=7+3=10$$

Therefore, F is selected next because it has the smallest $f(n)$.

Expand F:

For C:

$$g(C)=7+2=9$$

$$h(C)=0$$

$$f(C)=9+0=9$$

Since C is the goal, the search terminates.

Node Expansion Table

Step	Node	$g(n)$	$h(n)$	$f(n) = g(n)+h(n)$	Action
1	W	0	8	8	Expand
2	B	5	6	11	Expand
3	F	7	3	10	Expand
4	C	9	0	9	Goal reached

Thus, the optimal path is:

$W \rightarrow B \rightarrow F \rightarrow C$

Total path cost:

$$5+2+2=9$$

Complexity Analysis

For a general graph with branching factor (b) and solution depth (d):

- Time Complexity: $(O(b^d))$ in the worst case.
- Space Complexity: $(O(b^d))$.

With an efficient priority queue, practical performance can be significantly better when the heuristic provides good guidance.

Advantages and Limitations

Advantages:

- Finds the optimal path when the heuristic is admissible and consistent.
- Considers both actual and estimated costs.
- More efficient than uninformed search when a good heuristic is available.
- Widely applicable to route planning and navigation problems.

Limitations:

- Can require substantial memory.
- Performance depends on the quality of the heuristic.
- A poor heuristic can cause many unnecessary node expansions.
- An inadmissible heuristic can cause A* to return a non-optimal path.

Conclusion

A* Search determines the optimal delivery route by evaluating:

For the given network, the robot follows:

$$f(n)=g(n)+h(n)$$

$$W \rightarrow B \rightarrow F \rightarrow C$$

with a minimum total cost of 9. A* guarantees optimality when the heuristic is admissible and, in graph-search implementations, appropriately consistent. Humanity has apparently invented roads, weighted graphs, and then made robots calculate which road to take. At least the mathematics is useful.

3. A university needs to prepare an examination timetable without scheduling conflicting subjects at the same time. Design the problem as a Constraint Satisfaction Problem (CSP) by identifying variables, domains, and constraints. Explain how MRV and Forward Checking improve the efficiency of the solution.

Problem Statement

A university needs to prepare an examination timetable such that no two subjects having a common student are scheduled in the same time slot. The timetable is formulated as a Constraint Satisfaction Problem (CSP) by defining the subjects as variables, available examination slots as domains, and scheduling conflicts as constraints.

Problem Analysis

Consider five subjects:

- A = Artificial Intelligence
- D = Data Structures
- M = Mathematics
- N = Numerical Methods
- P = Programming

Assume the following subjects have common students:

- A conflicts with D and M
- D conflicts with A, M, and P
- M conflicts with A, D, and N
- N conflicts with M and P
- P conflicts with D and N

Assume there are three examination slots:

$T = \{T1, T2, T3\}$

The objective is to assign exactly one time slot to every subject while ensuring that conflicting subjects are never assigned the same slot.

AI Technique Selection

The problem is modeled as a Constraint Satisfaction Problem (CSP).

Two techniques are used to improve the CSP search:

- Minimum Remaining Values (MRV): Selects the variable with the smallest number of legal remaining values.
- Forward Checking: After assigning a value, removes that value from the domains of all neighbouring unassigned variables.
-

Justification of Algorithm

A CSP approach is suitable because the timetable consists of:

1. A fixed set of subjects.
2. A finite set of examination slots.
3. Constraints between subjects that cannot occur simultaneously.

MRV reduces unnecessary search by selecting the most restricted subject first, while Forward Checking detects conflicts early instead of waiting until a complete timetable has been constructed. Humans apparently needed algorithms to discover that scheduling two exams for the same students at the same time is a bad idea.

State Space Representation

Each state represents a partial timetable assignment.

For example:

```

      Start
      |
      A = T1
    /   \
  D=T2   D=T1
  |       |
  M=T3   Conflict
  |
  N=T1
  |
  P=T3

```

A goal state is reached when every subject has been assigned a valid examination slot without violating any constraint.

Variables, Domains, and Constraints

Variables

Each subject is a variable:

$X = \{A, D, M, N, P\}$

Domains

Initially, every subject can be assigned any of the three examination slots:

$D(A) = D(D) = D(M) = D(N) = D(P) = \{T1, T2, T3\}$

Constraints

Conflicting subjects must have different examination slots.

$A \neq D$

$A \neq M$

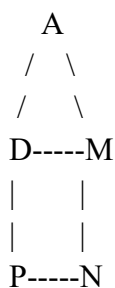
$D \neq M$

$D \neq P$

$M \neq N$

$N \neq P$

These constraints can also be represented as a constraint graph:



An edge between two subjects means that they cannot be scheduled at the same time.

Heuristic Function MRV acts

For CSPs, as the variable-selection heuristic.

$MRV(X)$ = variable with the fewest legal remaining values

If two or more variables have the same number of remaining values, a degree heuristic can be used as a tie-breaker by selecting the variable involved in the largest number of constraints.

Unlike informed graph search, there is no $h(n)$ calculation here. MRV is the relevant CSP heuristic.

Algorithm Design / Pseudocode

CSP_Timetable(Variables, Domains, Constraints)

1. If all variables are assigned:
 Return the complete timetable
2. Select an unassigned variable X using MRV.
3. For each value V in Domain(X)
 - a. If assigning $X = V$ satisfies all constraints:
 Assign $X = V$.
 Apply Forward Checking:
 Remove V from the domains of all

unassigned neighbours of X

If no neighbour has an empty domain:

 Recursively solve the remaining CSP.

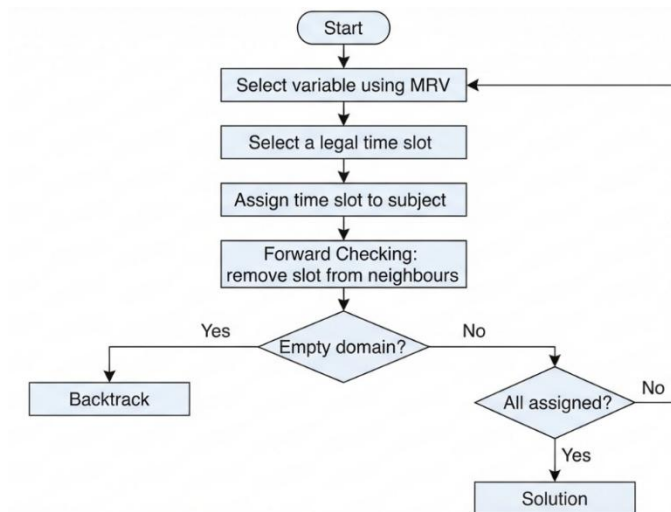
If solution is found:

 Return solution.

Undo $X = V$ and restore the domains.

4. Return failure.

Flowchart / Algorithm Diagram



Working / Execution Trace

Initially:

$A = \{T1, T2, T3\}$

$D = \{T1, T2, T3\}$

$M = \{T1, T2, T3\}$

$N = \{T1, T2, T3\}$

$P = \{T1, T2, T3\}$

All variables have three values, so MRV has a tie. Select D using the degree heuristic because it has several constraints.

Assign:

$D = T1$

Forward Checking removes T1 from the domains of its neighbours A, M, and P.

$A = \{T2, T3\}$

$D = \{T1\}$

$M = \{T2, T3\}$

$N = \{T1, T2, T3\}$

$P = \{T2, T3\}$

MRV now selects one of A, M, or P. Select A.

Assign:

$A = 2$

Forward Checking removes T2 from M.

$A = \{T2\}$

$D = \{T1\}$

$M = \{T3\}$

$N = \{T1, T2, T3\}$

$P = \{T2, T3\}$

MRV selects M, because its domain contains only one value.

Assign:

$M = T3$

Forward Checking removes T3 from N.

$N = \{T1, T2\}$

$P = \{T2, T3\}$

MRV selects N.

Assign:

$N = T1$

Since N conflicts with P, Forward Checking removes T1 from P. However, P already has:

$P = \{T2, 3\}$

so no conflict occurs.

Assign

$P = T2$

The final timetable is:

Subject	Examination Slot
A	T2
D	T1
M	T3
N	T1
P	T2

Verification:

- $A \text{ and } D \rightarrow T2 \neq T1 \checkmark$
- $A \text{ and } M \rightarrow T2 \neq T3 \checkmark$
- $D \text{ and } M \rightarrow T1 \neq T3 \checkmark$
- $D \text{ and } P \rightarrow T1 \neq T2 \checkmark$
- $M \text{ and } N \rightarrow T3 \neq T1 \checkmark$
- $N \text{ and } P \rightarrow T1 \neq T2 \checkmark$

Therefore, the timetable satisfies all constraints.

Complexity Analysis

For n subjects and k available examination slots, the worst-case search space is:

$[O(k^n)]$

With MRV and Forward Checking, many invalid assignments are eliminated before the complete search tree is explored, significantly reducing practical search effort.

The exact improvement depends on the structure of the constraint graph and the number of available slots.

Advantages and Limitations

Advantages:

- Naturally represents examination scheduling problems.
- MRV prioritizes the most constrained subject.
- Forward Checking detects impossible assignments early.
- Reduces unnecessary backtracking.
- Can accommodate additional scheduling constraints.

Limitations:

- Worst-case complexity remains exponential.
- Performance depends on the constraint structure.
- Large timetabling problems may require additional CSP techniques.
- A poorly chosen variable/value ordering can still cause considerable backtracking.
-

Conclusion

The university examination timetable can be effectively modeled as a CSP using subjects as variables, examination slots as domains, and shared-student conflicts as constraints. MRV selects the most constrained subject first, while Forward Checking eliminates invalid slot choices immediately. The resulting valid timetable is:

A=T2, D=T1, M=T3, N=T1, P=T2

Thus, the timetable is generated without scheduling any conflicting subjects in the same examination slot.

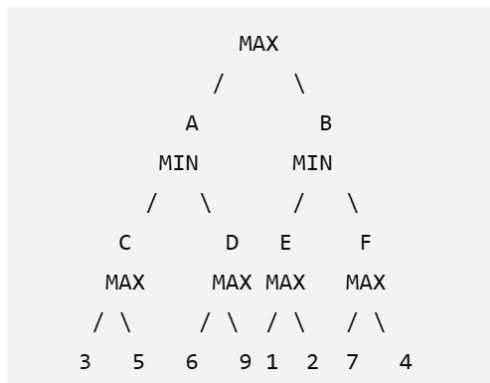
4. Design the Minimax algorithm for the given game tree to determine the best move for the MAX player. Then apply Alpha–Beta Pruning to the same tree and indicate which nodes are pruned. Compare the number of nodes evaluated before and after pruning

Problem Statement

Design the Minimax algorithm for a given game tree to determine the best move for the MAX player. Then apply Alpha–Beta Pruning to the same tree, identify the pruned nodes, and compare the number of nodes evaluated before and after pruning.

Problem Analysis

Consider the following game tree. MAX and MIN players alternate turns, and the leaf nodes contain utility values for MAX.



MAX wants to maximize the final value, while MIN wants to minimize it.
The objective is to determine whether MAX should choose A or B.

AI Technique Selection

The techniques selected are:

- Minimax: Determines the optimal move assuming both players play optimally.
- Alpha–Beta Pruning: Eliminates branches that cannot affect the final Minimax decision.

Justification of Algorithm

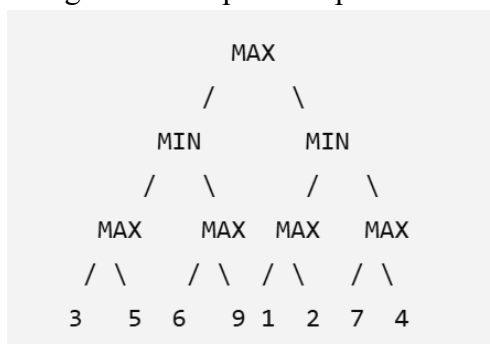
Minimax is appropriate for a two-player adversarial game because each player has an opposing objective.

- MAX selects the highest value.
- MIN selects the lowest value.

Alpha–Beta Pruning improves Minimax by avoiding unnecessary node evaluations while producing the same optimal result.

State Space Representation

The game tree represents possible moves from the current game state:



The root is the current state, and the leaf nodes represent terminal game states.

Variables, Domains, and Constraints

This is not a CSP problem, so variables, domains, and constraints are not applicable.

Heuristic Function

A heuristic function is not required because the complete game tree and terminal utility values are given. The utility values at the leaf nodes are directly used by Minimax.

Algorithm Design / Pseudocode

MINIMAX(node, maximizingPlayer)

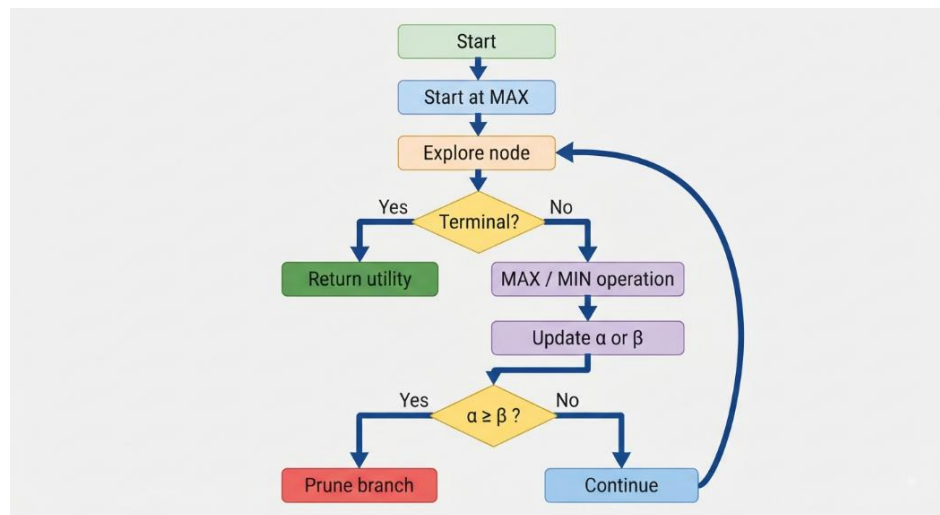
1. If node is a terminal node:
 return utility(node)
2. If maximizingPlayer:
 value = $-\infty$
 For each child of node:
 value = MAX(value,
 MINIMAX(child, FALSE))
 return value
3. Otherwise:
 value = $+\infty$
 For each child of node:
 value = MIN(value,
 MINIMAX(child, TRUE))
 return value

Alpha-Beta Pruning Algorithm

ALPHA-BETA(node, α , β , maximizingPlayer)

1. If node is a terminal node:
 return utility(node)
2. If maximizingPlayer:
 value = $-\infty$
 For each child:
 value = MAX(value,
 ALPHA-BETA(child, α , β , FALSE))
 α = MAX(α , value)
 If $\alpha \geq \beta$:
 break
 return value
3. Otherwise:
 value = $+\infty$
 For each child:
 value = MIN(value,
 ALPHA-BETA(child, α , β , TRUE))
 β = MIN(β , value)
 If $\alpha \geq \beta$:
 break
 return value

Flowchart / Algorithm Diagram



Working / Execution Trace

Minimax

Evaluate the left subtree A.

For C:

$$C = \max(3, 5) = 5$$

For D:

$$D = \max(6, 9) = 9$$

Since A is a MIN node:

$$A = \min(5, 9) = 5$$

Now evaluate the right subtree B.

For E:

$$E = \max(1, 2) = 2$$

For F:

$$F = \max(7, 4) = 7$$

Therefore:

$$B = \min(2, 7) = 2$$

At the root, MAX chooses:

$$\text{MAX} = \max(5, 2) = 5$$

Therefore:

Best Move=A

and the final Minimax value is: 5

Minimax Node Evaluation

All eight leaf nodes are evaluated:

3, 5, 6, 9, 1, 2, 7, 4

Therefore:

8 leaf nodes evaluate

Alpha–Beta Pruning

Nodes are evaluated from left to right.

Initially: $\alpha = -\infty$, $\beta = +\infty$

Evaluate A. C gives: $\max(3, 5) = 5$

Therefore, at A: $\beta = 5$

Now evaluate D.

The first child of D is 6. Since $6 \geq \beta (5)$, the remaining child 9 cannot affect the result. Therefore, 9 is pruned.

A therefore remains: $A = \min(5, 6) = 5$

At the root: $\alpha = 5$

Evaluate B. E gives: $\max(1, 2) = 2$

Therefore: $\beta = 2$

Since $\beta (2) \leq \alpha (5)$, the entire F subtree is pruned.

Therefore, both 7 and 4 are not evaluated.

The final result remains: $\text{MAX} = \max(5, 2) = 5$

Thus the best move is: A

Node Evaluation Comparison

Method	Evaluated Leaf Nodes	Pruned Nodes	Best Move	Final Value
Minimax	8	0	A	5
Alpha–Beta	5	3	A	5

Pruned nodes: 9, 7, 4

Reduction in nodes evaluated: $8 - 5 = 3$

Percentage reduction: $(3 / 8) * 100 = 37.5\%$

Thus, Alpha–Beta Pruning reduces the leaf-node evaluations by **37.5%** while producing exactly the same optimal decision.

Complexity Analysis

For branching factor (b) and tree depth (d):

Minimax:

$O(b^d)$

Alpha–Beta Pruning:

Worst case:

$O(b^d)$

Best case with excellent move ordering:

$O(b^{d/2})$

Therefore, Alpha–Beta Pruning can allow much deeper game trees to be searched within the same computational resources.

Advantages and Limitations

Minimax Advantages:

- Finds the optimal move when both players play optimally.
- Simple and systematic.
- Suitable for adversarial two-player games.

Minimax Limitations:

- Evaluates all relevant nodes.
- Computational cost increases rapidly with tree depth.

Alpha–Beta Advantages:

- Produces the same result as Minimax.
- Avoids unnecessary node evaluations.
- Can significantly improve search efficiency.

Alpha–Beta Limitations:

- Efficiency depends on move ordering.
- Worst-case complexity remains ($O(b^d)$).
- Does not change the underlying Minimax decision.

Conclusion

Minimax determines that the MAX player should choose move A, with a final utility value of: 5

Ordinary Minimax evaluates 8 leaf nodes. Alpha–Beta Pruning evaluates only 5 leaf nodes and prunes 3 nodes: 9, 7, and 4.

Thus, Alpha–Beta Pruning achieves the same optimal result while reducing the number of evaluated nodes by 37.5%.

5. A warehouse robot operates in a dynamic environment where obstacles may appear unexpectedly. Design an intelligent search strategy using either Local Search or an Online Search Agent. Explain how the algorithm adapts to environmental changes and compare its performance with informed search algorithms.

Problem Statement

Design an intelligent search strategy for a warehouse robot operating in a dynamic environment where obstacles can appear unexpectedly. The robot must continuously adapt its path based on newly observed information and reach the destination efficiently.

Problem Analysis

Consider a warehouse robot that must travel from a Start (S) position to a Goal (G) position. The warehouse contains shelves and obstacles that may change while the robot is moving. A traditional informed search such as A* assumes that the map is known and remains relatively stable. In a dynamic warehouse, this assumption can fail when a new obstacle blocks the planned path.

Therefore, an Online Search Agent is suitable because the robot:

1. Plans using the information currently available.
2. Moves through the environment.
3. Observes newly encountered obstacles.
4. Updates its internal map.
5. Recalculates the route when necessary.
6. Continues until the goal is reached.

AI Technique Selection

The selected technique is an Online Search Agent. The robot does not know the complete state of the environment beforehand. It discovers information while moving and uses that information to make subsequent decisions.

A simple heuristic can be used:

$h(n)$ = Manhattan distance from n to Goal

For grid movement: $h(n) = |x_n - x_G| + |y_n - y_G|$

Justification of Algorithm

An Online Search Agent is appropriate because the warehouse environment is partially known and dynamic. If an obstacle suddenly appears, the robot does not need to restart the entire process from the beginning. It updates the known environment and searches for an alternative route from its current position. This makes online search more practical for real-world robots than a purely offline search that assumes a fixed map.

State Space Representation

The warehouse can be represented as a grid.

For example:

S	.	.	.	.
.	X	X	.	.
.	.	.	.	.
.	X	.	X	.
.	.	.	.	G

Where:

- S = Starting position
- G = Goal
- . = Free cell
- X = Obstacle

Each grid position represents a possible state of the robot.

When the robot discovers a new obstacle, the state-space representation is updated.

Variables, Domains, and Constraints

This is not a CSP problem, so variables, domains, and constraints are not applicable.

Heuristic Function

Although Online Search does not require a fixed heuristic in the same way as an offline informed search, a heuristic can help the robot choose promising neighbouring states.

For a four-directional grid:

$$h(n) = |x_n - x_G| + |y_n - y_G|$$

The heuristic estimates the number of grid movements required to reach the goal.

For example, if the robot is at (2,3) and the goal is at (5,5):

$$h(n) = |2-5| + |3-5|$$

$$h(n) = 3 + 2 = 5$$

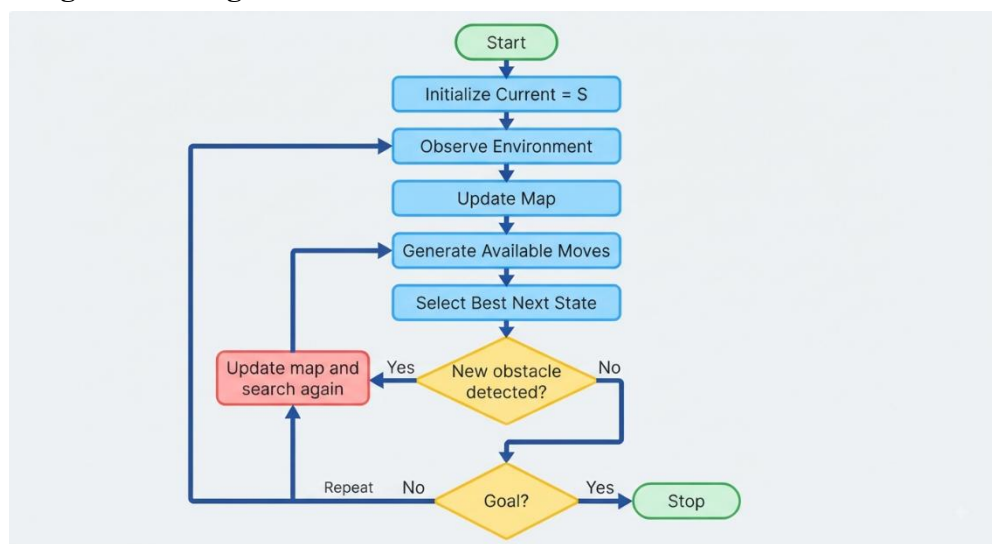
When an obstacle is detected, the robot recalculates its available moves and avoids the blocked state.

Algorithm Design / Pseudocode

OnlineSearchAgent(Start, Goal)

1. Current $\leftarrow$ Start
2. Create an initially unknown map.
3. Mark Current as known and free.
4. While Current $\neq$ Goal:
 - a. Observe the surrounding environment.
 - b. Update the map with newly detected obstacles and free cells.
 - c. Generate all reachable neighbouring states.
 - d. Remove blocked or unsafe states.
 - e. Calculate $h(n)$ for each available state.
 - f. Select the most promising reachable state.
 - g. Move the robot to the selected state
 - h. If a new obstacle blocks the planned route:
Update the map.
Recalculate the route from Current.
5. Return "Goal Reached".

Flowchart / Algorithm Diagram



Working / Execution Trace

Assume the robot initially plans:

$S \rightarrow A \rightarrow B \rightarrow G$

The robot begins following this route.

Initial Situation

$S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$

The robot moves

$S \rightarrow A$

Environmental Change

Suppose an obstacle suddenly appears at B.

The robot detects:

$S \rightarrow A \rightarrow X$

\

$D \rightarrow E \rightarrow G$

The robot marks B as blocked and updates its internal map.

Instead of continuing toward the blocked node, it evaluates alternative states from A and selects:

$A \rightarrow D$

It then continues:

$D \rightarrow G$

Therefore, the adapted route becomes:

$S \rightarrow A \rightarrow D \rightarrow E \rightarrow G$

The important feature is that the robot does not require complete prior knowledge of the environment. It discovers the obstacle and changes its route while operating.

Performance Comparison

Feature	Online Search Agent	A* Informed Search
Environment knowledge	Initially incomplete	Usually requires known map
Handles new obstacles	Yes	Requires map update and replanning
Adaptability	High	Moderate
Initial planning	Limited	Complete
Path quality	Depends on discoveries and decisions	Optimal with suitable heuristic
Memory requirement	Can be lower initially	Can be high
Suitable for dynamic environments	Excellent	Requires repeated replanning

An informed algorithm such as A* can be more efficient when the complete environment is known and stable. However, an Online Search Agent is better suited when the robot must discover and respond to changes while operating.

Complexity Analysis

For a grid with (N) reachable states:

- Worst-case search time: $O(N)$ for exploring the available state space.
- Space complexity: $O(N)$ for storing the discovered map and visited states.

In a dynamic environment, additional computation may be required whenever the environment changes because the robot must update its map and reconsider its route.

Advantages and Limitations

Advantages:

- Adapts to unexpected obstacles.
- Works with incomplete environmental information.
- Can replan during execution.
- Suitable for real-time robotic applications.
- Avoids relying completely on a pre-built map.

Limitations:

- The resulting path may not always be globally optimal.
- Frequent environmental changes can increase computation.
- Performance depends on the quality of observations.
- The robot may need to explore additional states when information is unavailable.

Conclusion

An Online Search Agent provides an effective strategy for a warehouse robot operating in a dynamic environment. The robot observes its surroundings, updates its internal map, detects unexpected obstacles, and dynamically selects an alternative route. Unlike conventional informed search, which generally relies on a known map, online search can make decisions while discovering the environment, making it more suitable for unpredictable warehouse conditions.